

Interface With Voice-Band Codecs Using I²C

Wendy X. Fang, Application Engineer

High Performance Analog Group

ABSTRACT

This application report builds up an I²C port from two pins in a McBSP, develops the I²C software driver, and interfaces with the voice-band codec(s), including TLV320AIC1103/ 1109 /1110, TLV320AIC12/ 13/ 14/ 15, or TLV320AIC20/ 21/ 24/ 25 devices. The I²C driver design is discussed in details and the corresponding C code is presented.

Contents

1	Introduction	2
2	I²C Port	2
3	I²C Program	4
3.1	4-Frames-Per-Bit Scheme	4
3.2	Start-Bit and Stop-Bit	7
4	Codec Configuration	8
4.1	Start Package	8
4.2	Address Package	10
4.3	Data Writing Package	11
4.4	I ² C Driver for CR Writing	12
5	Read Codec Control Registers	15
5.1	Address Package	15
5.2	Restart Package for Reading	16
5.3	Data Reading Package	16
5.4	I ² C Driver for CR Reading	17
6	References	18

List of Figures

1	4-Frame-Per-Bit Scheme to Assure the Proper I ² C Timing	4
2	I ² C Start- and Stop-Bits	7
3	I ² C Start Package for Writing	9
4	I ² C CR Address Package for Writing	10
5	I ² C Configuration Data Package (for writing and reading)	11
6	I ² C CR Address Package for Reading	15

List of Tables

1	All Control Registers of an AIC20 Codec Channel	18
---	---	----

TMS320C5000 and TMS3206000 are trademarks of Texas Instruments.

1 Introduction

Generally, to apply a codec always involves the *setting-up* or software configuration of the codec. For a TLV320AIC1103/1109/1110 PCM codec, a 2-pin I²C port provides the primary interface port for the configuration; while for a TLV320AIC12/13/14/15 or TLV320AIC20/21/24/25 codec, the configuration can be performed either through its 4-wire data port or its 2-wire control port, in I²C or S²C protocol. This application report discusses only the codec configuration using the I²C protocol.

Not all host processors, such as DSPs or MCUs, have the on-chip built-in I²C port. In this case, two GPIO pins are used, with proper software/firmware driver, to implement the I²C interface and to communicate with other I²C digital devices. Most TI's DSPs, such as that in the TMS320C5000™ and TMS320C6000™ DSP families, have more than one multi-channel buffered serial port (McBSP). In addition to its common serial interface function, the pins in a McBSP can also be programmed as GPI, GPO, or GPIO. This application report is utilizing two of the McBSP's pins, programmed as GPO and GPIO pins respectively, to interface and configure a codec through the I²C bus.

The basic design concept is discussed in details, together with the associated C code. The I²C drivers have been developed, debugged and implemented on TI's voice-band codec development platform with a TLV320AIC1110, a TLV320AIC12, or two (2) TLV320AIC20 (4 codec channels) devices on its or their EVM board respectively, interfacing with TI's TMS320C54xx DSK board. The corresponding application code is given in the zip files found in the product folder on the TI web page.

2 I²C Port

The I²C port is built up using two McBSP's pins. Disable the McBSP's serial port function so that all of the seven (7) McBSP pins become the general-purpose input (GPI), output (GPO), or input-or-output (GPIO) pins. This application report programs the MCBSP0 and uses the CLKX0 pin as the SCL of the I²C port; and the FSX0 pin as the SDA. Where the McBSP's data pin, DX, or DR, are not used for SDA since SDA is bidirectional (as input or output) while a McBSP's data pin is single directed, that is: the DX pin can only be used as GPO and the DR pin can only be used as GPI.

For the C54xx DSP, to program the CLKX0 pin as the GPO and FSX0 pin as the GPIO, set up the followings:

```
McBSP0_SPSA = SPCR1;
McBSP0_SPSD = 0x0000;    // McBSP0 receiver is disabled and reset

McBSP0_SPSA = SPCR2;
McBSP0_SPSD = 0x0000;    // McBSP0 transmitter is disabled and reset

McBSP0_SPSA = PCR;
McBSP0_SPSD = 0x3A0A;    // all McBSP0 pins are GPIO/GPI/GPO
                        // both CLKX and FSX are outputs, & CLKX=FSX = 1
```

In the above code, the McBSP0_SPSA and McBSP0_SPSD are the memory mapped registers at the DSP's IO space 0x0038 and 0x0039 respectively; and the PCR is the McBSP's control register and located at the subaddress 0x000E.

Both CLKX and FSX can be GPIO pins. Through the above setup, the CLKX (SCL) and FSX (SDA) are output pins for now; and their outputs are both logic high, which is the exact initial condition for an I²C port. From this point the CLKX pin is called the SCL and FSX is called SDA.

The SCL should always be an output from the host processor, so that its direction does not change. The SCL should output either 1 or 0 and functions as the interface clock. To do so, we define the macros: Set_SCLHi() and Set_SCLLo(), and where:

```
#define Set_SCLHi( )    McBSP0_SPSA = PCR; McBSP0_SPSPD |= 0x0002
#define Set_SCLLo( )    McBSP0_SPSA = PCR; McBSP0_SPSPD &= 0xFFFFD
```

The I²C data pin SDA must be an output pin for writing or an input pin for reading. Therefore, its direction must be able to change. To change SDA pin's direction, Set_SDADirOut() and Set_SDADirIn() are defined as:

```
#define Set_SDADirOut( )    McBSP0_SPSA = PCR; McBSP0_SPSPD |= 0x0800
#define Set_SDADirIn( )    McBSP0_SPSA = PCR; McBSP0_SPSPD &= 0xF7FF
```

The SDA should be either 1 or 0, depending on the corresponding bit of the data. To output or write 1 or 0, we have the Set_SDAHi() and Set_SDA Lo() macros:

```
#define Set_SDAHi( )    McBSP0_SPSA = PCR; McBSP0_SPSPD |= 0x0008
#define Set_SDA Lo( )    McBSP0_SPSA = PCR; McBSP0_SPSPD &= 0xFFFF7
```

Besides the direction and logic status of the SCL and SDA, to run the I²C protocol, the SCL clock frequency should be within a certain range per I²C bus specification [1]. The I²C on the TLV320AIC1103/ 1106/ 1110 codec, for example, can be run at the fast mode, i.e.: the SCL rate can be from 0 Hz and up to 400 kHz. To control the SCL frequency, this application report makes use of the TIMER0 in the C54xx DSP. Program and enable TIMER0, use the timer to control the clock frequency of the SCL. The followings are two examples:

Example 1: For C5402 DSP, if CPU clock = 100 MHz, using the following code to program TIMER0 rate 16 kHz:

```
TCR = 0x0010;    // stop TIMER0 and TDDR=0
PRD = 6249;      // TIMER0 rate = CPU-Frequency/(PDR+1) = 100MHz/6250 = 16kHz
...
TCR &= 0xFFEF;   // start TIMER0
...
```

Example 2: For C5416 DSP, if CPU clock = 160 MHz, using the following code to program TIMER0 rate 16 kHz:

```
TCR = 0x0010;    // stop TIMER0 and TDDR=0
PRD = 9999;      // TIMER0 rate = CPU-Frequency/(PDR+1) =160 MHz/10000 = 16 kHz
...
TCR &= 0xFFEF;   // start TIMER0
...
```

Each time the TIMER0 returns to zero and the MMR PRD is reloading, the TIMER0 interrupt flag sets to logic 1, even though the interrupt is not enabled. By checking the flag (in the MMR IFR), the SCL rate can be controlled. For more details, refer to the code in the zip files found in the product folder on the TI web page.

3 I²C Program

In order to have the reliable I²C communication, this application note uses a *four (4) frames per bit* scheme, so that the line delays and noise have less effect on the interface.

3.1 4-Frames-Per-Bit Scheme

The 4-frames-per-bit scheme assures the SDA does not change at the SCL's changing edges, and only allows data changing at frame#0, and read data only at frame #2, which is described in Figure 1. As seen in the Figure 1, the SCL changes at Frame#1 and #3, and the SDA changes at Frame#0 and is read at Frame#2. In this scheme, the SDA is already stable when the SCL is changing and the codec devices are writing/reading the data through the I²C bus.

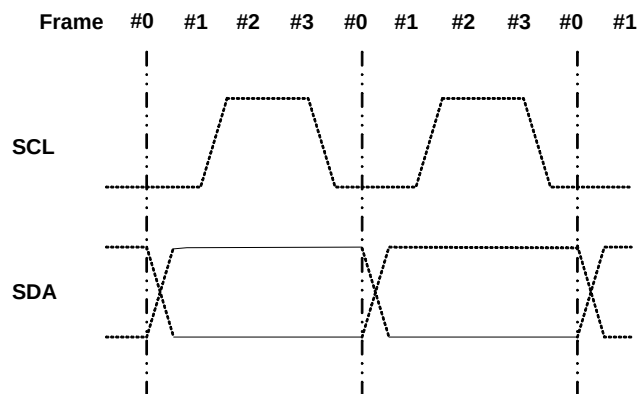


Figure 1. 4-Frame-Per-Bit Scheme to Assure the Proper I²C Timing

Built on the 4-frames-per-bit scheme, the corresponding bit-writing program can be:

```
void I2CWrite(unsigned int WriteBit)
{
    Set_SDADirOut( );           // set SDA as output for writing
    switch(FrameCount)
    {
        case(0):                // at the first frame
            Set_SCLLo( );        // SCL remains 0
            if (WriteBit == 0)    // SDA = WriteBit
                Set_SDA Lo( );
            else
                Set_SDAHi( );
            break;
        case(3):                // at the last (4th) frame
            Set_SCLLo( );        // SCL back to 0
            break;
        default:                 // at the 2nd & 3rd frames
            Set_SCLHi( );        // SCL is 1
    }
    FrameCount += 1;            // control frame counter from 0 to 3
    if (FrameCount > 3)
```

```

    {
        FrameCount = 0;           // after four frames, 1 bit has been sent out
        BitIndex = (BitIndex >> 1); // reset FrameCount
    }
}

```

And, similarly, the program for reading is:

```

void I2CRead ( )
{
    Set_SDADirIn( );           // set SDC as input for reading
    switch(FrameCount)
    {
        case(1):                // at the 2nd frame
            Set_SCLHi( );       // SCL goes high
            break;
        case(2):                // at the 3rd frame
            Set_SCLHi( );       // SCL remains high
            if (Get_SDA( ) == 1) // set corresponding bit when reading 1
                CRContent[CRIndex] |= BitIndex
            break;              // else the bit remains at 0
        default:                // at the 1st and last frames
            Set_SCLLo( );       // SCL is 0
    }
    FrameCount += 1;           // control frame counter from 0 to 3
    if (FrameCount > 3)
    {
        // after four frames, 1 bit has been read in
        FrameCount = 0;        // reset FrameCount
        BitIndex = (BitIndex >> 1); // Mask to the next reading bit
    }
}

```

In the above code, the lower byte (8 bits) of the CRContent[CRIndex] is used to store the 8 bits data and it should be initialized to 0x0000 before starting to read; the BitIndex is the corresponding bit mask and it should be shifted right after each bit reading; and the subroutine Get_SDA() reads the logic status of the SDA (input) pin, and returns a 0 or 1, i.e.:

```

Unsigned int Get_SDA( )
{
    McBSP0_SPSA = PCR;
    return ( (McBSP0_SPSP >> 3) & 0x0001 );
}

```

The acknowledge bit ACK has the 4-frames per bit, also. Note that, the ACK is an input after writing and an output after reading. Therefore, the direction of the SDA is different correspondingly. Two I²C ACK subroutines are defined to handle the ACK after a writing, I2CWACK (); and after reading, I2CRACK ().

```

void I2CWACK ( )
{
    Set_SDADirIn ( );           // ACK bit after a writing
    switch ( FrameCount )       // set SDA direction as an input
    {                           // generate SCL in the 4 frames
        case (0):
            Set_SCLLo ( );
            break;
        case (3):
            Set_SCLLo();
            break;
        default:
            Set_SCLHi ( );
    }
    FrameCount += 1;           // control the 4 frames per bit
    If ( FrameCount > 3 )
        FrameCount = 0;
}

void I2CRACK ( )
{
    Set_SDADirOut ( );         // ACK bit after a reading
    Set_SDALo ( );             // set SDA direction as an output
    switch ( FrameCount )      // acknowledge
    {                           // generate SCL in the 4 frames
        case (0):              // and generate the ACK bit
            Set_SCLLo ( );
            break;
        case (3):
            Set_SCLLo();
            break;
        default:
            Set_SCLHi ( );
    }
    FrameCount += 1;           // control the 4 frames per bit
    If ( FrameCount > 3 )
        FrameCount = 0;
}

```

3.2 Start-Bit and Stop-Bit

Unlike other bits, the I²C start-bit and stop-bit are generated in three frames, as Figure 2 illustrates.

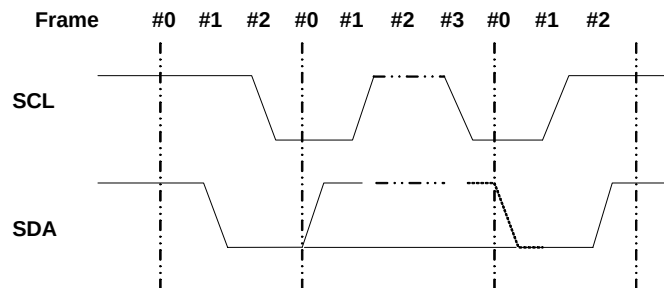


Figure 2. I²C Start- and Stop-Bits

The code for generating the start-bit and stop-bit are, respectively:

```
void I2CSTA ( )
{
    Set_SDADirOut ( );           // I2C start-bit
    switch ( FrameCount )        // SDA is an output
    {
        case (1):                // the 2nd frame
            Set_SCLHi ( );
            Set_SDA Lo ( );
            break;
        case (2):                // the last (3rd) frame
            Set_SCLLo ( );
            Set_SDA Lo ( );
            break;
        default:                  // the first frame
            Set_SCLHi ( );
            Set_SDA Hi ( );
    }
    FrameCount += 1;             // control frame counter from 0 to 2
    if ( FrameCount > 2 )
    {
        FrameCount = 0;
        BitIndex = 0x0080;      // mask the MSB of the low 8 bits
    }
}
```

```

void I2CST0 ( )
{
    // I2C start-bit
    Set_SDADirOut ( );           // SDA is an output
    switch ( FrameCount )
    {
        case (0):                // the first frame
            Set_SCLLo ( );
            Set_SDALo( );
            break;
        case (1):                // the last 2nd frame
            Set_SCLHi ( );
            Set_SDALo( );
            break;
        default:                 // the (3rd)frame
            Set_SCLHi ( );
            Set_SDAHl ( );
    }
    FrameCount += 1;             // control frame counter from 0 to 2
    if ( FrameCount > 2 )
    {
        FrameCount = 0;
        PackCount += 1;         // increase package counter
    }
}

```

In the above I²C stop code, the PackCount is the counter of the data package, and is discussed more in the next section.

4 Codec Configuration

The codec configuration instructs the host processor to write to the codec's control registers through the I²C bus.

For simplification, this application report arranges the I²C writing process in *packages*. Each package is either 9 bits or 10 bits long. The 10 bits long package must be either a start-package or a stop-package.

4.1 Start-Package

A *start-package* is also called the *device address package*, including: a start bit; 7-bit device address; a reading/writing bit; and an ACK bit. Figure 3 shows the start package for codec configuration.

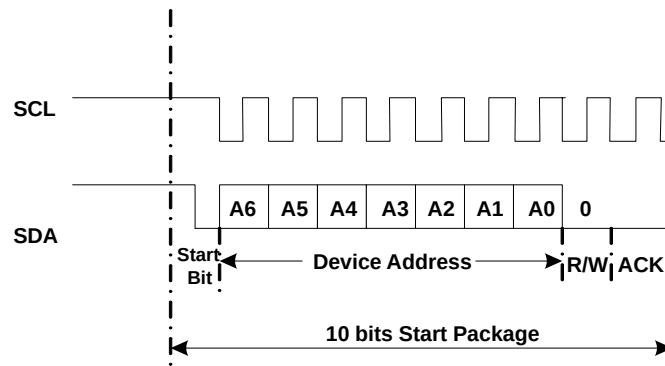


Figure 3. I²C Start Package for Writing

Normally, besides the start bit and the ACK bit, the other 8 bits can be expressed by a hex data, and named the *DeviceAddr* in this application report. The LSB is the R/W bit. When the LSB is 0, it requests a *writing*; and when it is 1, it requests a *reading*. For the AIC03/09/10 codec, the *DeviceAddr* is fixed as 0xE2 for writing or 0xE3 for reading. For the AIC12/13/14/15 and AIC20/21/24/25 codec, the 7 MSB bits in the *DeviceAddr* consists of two parts: part 1 is called the I²C device address and it includes the bits A6 to A4, refer to Figure 3, which is programmable through the codec's control register 2 (CR2) and defaulted as 100b; part 2 is named as the SMARTDM device address and includes the bits A3 to A0, refer to Figure 3. Also refer to the data sheet for the SMARTDM device address assignment. Here are some examples (with default I²C device address 100b): a stand alone AIC12 device address is 0000b and, then, the *DeviceAddr* is 0x80 for the writing operation. Another example: there are 8 AIC24 devices cascaded and, then, the first codec channel in the master device is of the *DeviceAddr* 0x9E, the next channel 0x9C, then 0x9A, 0x98, 0x96, 0x94, 0x92, 0x90, 0x8E, 0x8C, 0x8A, 0x88, 0x86, 0x84, 0x82, and the last slave codec 0x80.

The code for the sending or writing the start package is:

```
...
if (PackCount == 0 )                                // send package 0 (start
                                                        package)
{
    if (LoopCount < 4)                                // start bit
        I2CSTA ( );
    else if ( (LoopCount >= 4) && (LoopCount <36)) // 8-bits device address
        I2CWrite ( DeviceAddr & BitIndex );
    else if ( (LoopCount >= 36) && (LoopCount <40) // ACK bit
        I2CWACK ( );
    else
    {
        // after the ACK
        LoopCount = 0;                                // clear loop count
        PackCount = 1;                                // exit package 0 (to package 1)
        BitIndex = 0x0080;                            // set MSB for next package
    }
}
...
```

4.2 Address Package

The address in the *address-package* indicates the memory address of the device, its address given in the start-package. In this application report, the address package delivers the codec's control register (CR) address. For I²C writing, the address package is 9 bits long, including: 8 bits CR address; and an ACK bit. Figure 4 shows the address package.

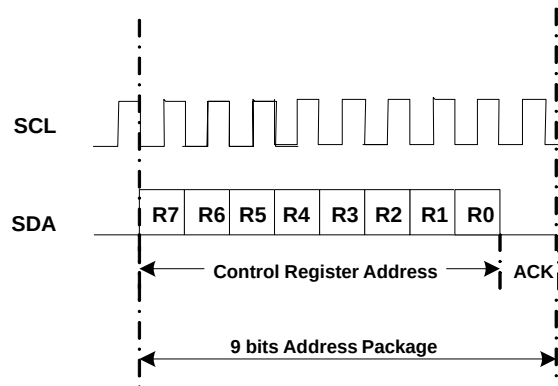


Figure 4. I²C CR Address Package for Writing

In the AIC03/09/10 codec, the CR address is 8 bits, from 0x00 to 0x06 for the 7 CRs. In the AIC12/13/14/15 and AIC20/21/24/25 codec, the 8 bit CR address consists of two parts: part 1 has 5 bits, R7 to R3, and can be either 00000b for nonbroadcast mode, or 11111b for broadcast mode; part 2 includes R2 to R0, and can be 001b (CR1), 010b (CR2), 011b (CR3), 100b (CR4), 101b (CR5), or 110b (CR6).

The code for the sending or writing address package is:

```
...
else if (PackCount == 1 )                               // send package 1 (start package)
{
    if ( (LoopCount >= 32)                                // 8-bits device address
        I2CWrite ( CRAddress & BitIndex );
    else if ( (LoopCount > 32) && (LoopCount <= 36)        // ACK bit
        I2CWACK ( );
    else
    {
        // after the ACK
        LoopCount = 0;                                     // clear loop count
        PackCount = 2;                                     // exit package 1 (to package 2)
        BitIndex = 0x0080;                                 // set MSB for next package
    }
}
...

```

For the AIC1103/09/10 codec, the CRAddress in the above code is simply the CR address from 0x00 to 0x06. And it needs to be updated for configuring different CRs. For the AIC12/13/14/15 or AIC20/21/24/25 codec, however, a sequence of CR can be written one after another. So the CRAddress is used to indicate the first or starting CR's address. The first data is written to the CR with the address = CRAddress; the next data is written to the next CR, and so on. For example: if CRAddress = 0x01, the data sent in the next package goes to the CR1; then the next data goes to the CR2; and so on, not needing to start or stop package in between.

4.3 Data Writing Package

The *data-writing-package* sends the configuration data to the corresponding CR with the address or starting address given in the previous packages. For I²C writing, the data writing package is either 9 bits long, if not the last data package; or 10 bit long, if it is the last data package. Including: 8 bits configuration data; and an ACK bit, (and the stop-bit if the last data package). Figure 5 shows the data package.

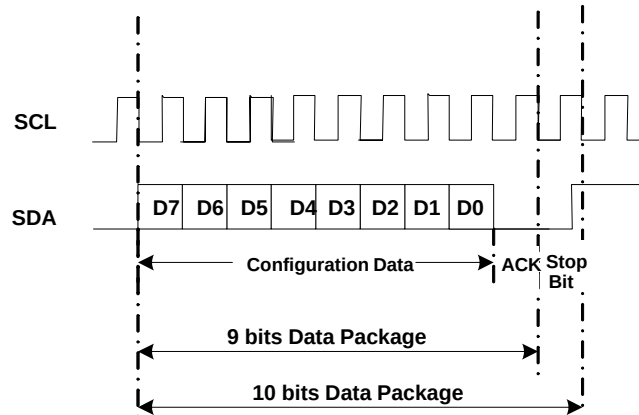


Figure 5. I²C Configuration Data Package (for writing and reading)

In the AIC03/09/10 codec, the data package is always 10 bits, with the last bit the stop-bit. The corresponding code for the data writing package is:

```
...
else if (PackCount == 2 )           // send package 2 (data package)
{
    if ( (LoopCount <= 32)           // 8-bits configuration data
        I2CWrite (ConfgData[CRIndex] & BitIndex);
    else if ((LoopCount > 32) && (LoopCount <=36) // ACK bit
        I2CWACK ( );
    else
        I2CST0 ( );                 // stop bit
}
...
```

In the AIC12/12/14/15 and AIC20/21/24/25 codecs, after the ACK, the routine should check for the next CR with the sequential address and determine if it must be configured or not. If yes, go to the next CR; otherwise, insert the stop-bit. The code could be:

```

...
else if (PackCount == 2 )           // send package 2 (data package)
{
    if ( (LoopCount <= 32)           // 8-bits configuration data
        I2CWrite (ConfigData[CRIndex] & BitIndex);
    else if ((LoopCount > 32) && (LoopCount <=36) // ACK bit
        I2CWACK ( );
    else if ((CRIndex+1) < (NumCRs)    // check if the last CR
        {                             //NOT: back to next data
            CRIndex += 1;
            LoopCount = 0;
            BitIndex = 0x0080;
        }
    else                             // YES:
        I2CST0 ( );                 // stop bit
}
...

```

4.4 I²C Driver for CR Writing

When configuring the CRs of one or more codecs, the three packages always need to get the I²C codec driver. That is:

```

void I2C_ConfigCodec (void)
{
    LoopCount += 1;
    if (PackCount == 0)
    {
        ...
    }
    else if (PackCount == 1)
    {
        ...
    }
    else if (PackCount == 2)
    {
        ...
    }
    else
    {
        PackCount = 1;
        LoopCount = 0;
        BitIndex = 0x0080;
        CRIndex += 1;
    }
}

```

For the different codec(s), the parameters in the driver may be different. Here are some examples.

Example 1: Configuring all CRs in an AIC1110 codec.

For configuring all 7 CRs in an AIC1110 codec, 7 I²C start/stop cycles are needed to configure the 7 CRs, and each time only one CR is configured. Or:

```

...
DeviceAddr = 0xE2;                // AIC1110 I2C device address
CRAddress [7] = { 0, 1, 2, 3, 4, 5, 6}; // CR start address is 0x00
ConfigData [7] = {                // default for PWRUPSEL = high
                                0x9B
                                0x02
                                0x40
                                0xA0
                                0x00
                                0x00
                                0x00
                                }

CRIndex    = 0;
BitIndex   = 0;
LoopCount  = 0;                // reset LoopCount
FrameCount = 0;                // reset FrameCount
PackCount  = 0;                // start from package 0
StartCR    while (CRIndex < 7)
{
    I2C_ConfigCodec(CRAddress[CRIndex]); // configure one CR a time
    LookControl( );                     // back to StartCR with
                                        control rate
}
...

```

Example 2: Configuring CR1, CR2, CR3, CR4(M), CR5A and CR6B in an AIC12 codec.

For configuring the six CRs in an AIC12 codec, only one start/stop I²C cycle is needed and the six CRs are configured one by one until the CRIndex reaches the NumCRs and the stop-bit is inserted.

```

...
DeviceAddr = 0x80;                // an standalone AIC12 device
                                   address
CRAddress = 0x01;                // CR start address is 0x01 pr
                                   CR1
ConfigData [6] = {0x61, 0xA0, 0x01, 0x84, 0x2A, 0x18};
CRIndex    = 0;
BitIndex   = 0;
LoopCount  = 0;                  // reset LoopCount
FrameCount = 0;                  // reset FrameCount
PackCount  = 0;                  // start from package 0
StartCR    while (CRIndex < 6)   //start from package 0
{
    I2C_ConfigCodec(CRAddress[CRIndex]); // configure one CR a time
    LookControl( );                     // back to StartCR with
                                         control rate
}
...

```

5 Read Codec Control Registers

With the very similar method, the Read-codec-CRs driver was developed. Note that the I²C reading includes one more package than the writing, a total of four packages. This application report only discusses the I²C reading protocol B, refer to the data sheet(s) of the codec device(s). Where the four packages consist of (1) the start package, the same as the start package for writing; (2) followed by the CR address package, the same as the CR address package for writing but plus a stop-bit; (3) the next is the restart package for reading, the same as (1) but the R/W bit is not 0 but 1 for reading; and (4) data reading. Because the first package is exactly the same as the writing, only three packages are discussed here.

5.1 Address Package

Figure 6 shows the address package for reading. Compared to writing, the only difference is that a stop-bit is added so that it is 10 bit long.

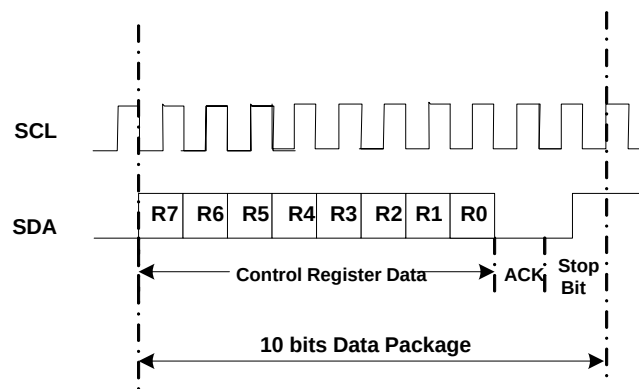


Figure 6. I²C CR Address Package for Reading

The code for the address package can be:

```
...
else if (PackCount == 1 )                                // send package 1 (address
                                                            package)
{
    if ( (LoopCount <= 32)                                // 8-bits device address
        I2Cwrite (CRAddress & BitIndex);
    else if ((LoopCount > 32) && (LoopCount <=36)          // ACK bit
        I2CWACK ( );
    else if ((LoopCount > 36) && (LoopCount < 40)          // stop bit
    {
        I2CSTO ( );
        PackCount = 1;
    }
    else
    {
        LoopCount = 0;                                    // after the ACK
        PackCount = 2;                                    // clear loop count
        BitIndex = 0x0080;                                // exit package 1 (to package 2)
        // set MSB for next package
    }
}
...
```

5.2 Restart Package for Reading

A *Restart-package* for reading is the same as the start-package for writing, Figure 3, except the R/W bit is 1. Therefore, the code for the sending or writing start package can be used for reading, with only very minor changes:

```

...
else if (PackCount == 2 )
{
    if ( (LoopCount 4)                                // start bit
        I2CSTA ( );
    else if ((LoopCount >= 4) && (LoopCount < 36)         // 8-bits device address
        I2CWrite ((DEviceAddr+1) & BitIndex) ;         // read from the device
    else if ((LoopCount >= 36) && (LoopCount <40)         // ACK bit
        I2CWACK ( );
    else
    {
        LoopCount = 0;                                // after the ACL
        PackCount = 3;                                // clear loop count
        BitIndex = 0x0080;                             // exit package 1 (to package 2)
        CRContent[CRIndex] = 0;                        // set MSB for next package
        CRContent[CRIndex] = 0;                        // clear up the space for reading
    }
}
...

```

5.3 Data Reading Package

The *data reading-package* reads the contents of the CR(s) with the start address given in the address package and the device indicated in the start package. Similar to writing, the data reading package is either 9 bits long, if not the last data package; or 10 bit long, if last data package. Including: 8 bits configuration data; and an ACK bit, (and the stop-bit if the last data package). Also refer to Figure 5. Note that the ACK bit here is an output to the codec(s) each time after reading an 8-bit data, i.e.: I2CRACK ().

The code for the data reading package can be, for AIC1110 codec:

```

...
else if (PackCount == 3 )                                // send package 2 (data package)
{
    if ( (LoopCount <= 32)                                // 8-bits CR content
        I2CRead ( );
    else if ((LoopCount > 32) && (LoopCount <= 36)         // ACK bit
        I2CRACK ( );
    else
        I2CSTO ( );                                     // stop bit
}
...

```


5.4 I²C Driver for CR Reading

Similar to the CR writing code, the CR reading can be performed and saved to the array CRContent using the routine I2C_ReadCodec().

```
void I2C_ReadCodec (void)
{
    LoopCount += 1;
    if (PackCount == 0)
    {
        ...
    }
    else if (PackCount == 1)
    {
        ...
    }
    else if (PackCount == 2)
    {
        ...
    }
    else if (PackCount == 3)
    {
        ...
    }
    else
    {
        PackCount = 1;
        LoopCount = 0;
        BitIndex = 0x0080;
        CRIndex += 1;
    }
}
```

The following is an example for codec's CR reading.

Example 3: Read all CRs in two AIC20 devices, four codec channels. Note that a codec channel in the AIC20 device has six registers, but some of the registers are multiple registers, as shown in the Table 1. Therefore, there are 14 CRs for an AIC20 channel. Also, there are four codec channels and there are a total of 54 (14x4) readings needed to read back all of the CRs in the system.

Table 1. All Control Registers of an AIC20 Codec Channel

CONTROL REGISTER	MULTIPLE OR SINGLE	CONTROL REGISTER NAME	READING CYCLE NO.
1 (001b)	Single	CR#1	1
2 (010b)	Single	CR#2	1
3 (011b)	Multiple	CR#3A CR#3B CR#3C CR#3D	1 2 3 4
4 (100b)	Multiple	CR#4 (for M) CR#4 (for N and P)	4 5
5 (101b)	Multiple	CR#5A CR#5B CR#5C CR#5D	5 6 7 8
6 (110b)	Multiple	CR#6A CR#6B	8 9

For reading back the contents in the 14 CRs in one AIC20 codec channel, this application report takes nine complete reading cycles, as indicated in the last column of Table 1. That is, the consequence readings can be done with AIC20 device, the first reading cycle reads three CRs, CR#1, CR#2 and CR#3A, note that CR3 is a multiple CR. In the second ready cycle, ask to read CR#3 again and the returned result is the content of a CR#3B. In the third cycle, ask to read CR#3 once more and the returned value is that in the CR#3C. In the fourth cycle, ask to read CR#3 and 2 CRs, then CR#3D and CR4 (M, or N, and P) is read. Ask to read CR4 again in the next cycle, the fifth cycle, and read 2 CRs, the contents in CR4(N and P, or M) and CR#5A is returned. In this way, all 14 CR contents in the codec channel's registers are read and stored consequently. Very often, the order (A, B, ...) of a multiple CR may shift, but they are consequential and all are stored together, one by one.

The above process must be repeated four times for the four codec channels, and a total of 36(4x9) reading cycles to read back all contents in the 56 CRs. The accrual code is given in the zip files found in the product folder on the TI web page.

6 References

1. The I²C-Bus Specification, Version 2.1, Jan. 2000. Download from Philips I²C website.
2. TLV320AIC1110 PCM Codec data manual, (SLAS359)
3. TLV320AIC12 data manual, (SLWS115)
4. TLV320AIC20 data manual, (SLWS363)
5. TMS320C54x DSP CPU and Peripherals, (SPRU131)
6. TMS320C54x DSP Enhanced Peripheral, (SPRU302)

IMPORTANT NOTICE

Texas Instruments Incorporated and its subsidiaries (TI) reserve the right to make corrections, modifications, enhancements, improvements, and other changes to its products and services at any time and to discontinue any product or service without notice. Customers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. All products are sold subject to TI's terms and conditions of sale supplied at the time of order acknowledgment.

TI warrants performance of its hardware products to the specifications applicable at the time of sale in accordance with TI's standard warranty. Testing and other quality control techniques are used to the extent TI deems necessary to support this warranty. Except where mandated by government requirements, testing of all parameters of each product is not necessarily performed.

TI assumes no liability for applications assistance or customer product design. Customers are responsible for their products and applications using TI components. To minimize the risks associated with customer products and applications, customers should provide adequate design and operating safeguards.

TI does not warrant or represent that any license, either express or implied, is granted under any TI patent right, copyright, mask work right, or other TI intellectual property right relating to any combination, machine, or process in which TI products or services are used. Information published by TI regarding third-party products or services does not constitute a license from TI to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property of the third party, or a license from TI under the patents or other intellectual property of TI.

Reproduction of information in TI data books or data sheets is permissible only if reproduction is without alteration and is accompanied by all associated warranties, conditions, limitations, and notices. Reproduction of this information with alteration is an unfair and deceptive business practice. TI is not responsible or liable for such altered documentation.

Resale of TI products or services with statements different from or beyond the parameters stated by TI for that product or service voids all express and any implied warranties for the associated TI product or service and is an unfair and deceptive business practice. TI is not responsible or liable for any such statements.

Mailing Address:

Texas Instruments
Post Office Box 655303
Dallas, Texas 75265